



中南大學

CENTRAL SOUTH UNIVERSITY

计算机视觉前沿技术

CSE

计算机学院

PyTorch基本概念

主讲： 沈成超

邮箱： scc.cs@csu.edu.cn

2026.04.24

目录



1. **Pytorch总览**
2. **核心基础：张量**
3. **核心模块：构建神经网络**
4. **核心机制：自动求导**
5. **总结与拓展**

1. Pytorch总览: What

Pytorch

- 由Meta AI开源的Python优先主流深度学习框架
- 基于 Torch 框架重构, 官方生态覆盖视觉、文本、音频全场景
- 一个不用从零写深度学习底层代码, 快速搭建、训练、部署AI模型的Python工具库



1. Pytorch总览: What

- 张量加速引擎
替代NumPy, 支持GPU/多硬件加速的多维数组 (张量) 运算
- 自动微分系统
内置Autograd自动求导引擎, 自动完成反向传播
- 神经网络库
torch.nn模块封装了所有常用网络层、损失函数、模型容器



1. Pytorch总览: Why

- Python原生语法
和普通Python代码逻辑完全一致, 调试方便
- 动态计算图
运行时构建计算图, 无需提前定义完整网络, 兼容Python控制流
- 强大兼容
兼容CPU、NVIDIA/AMD/ 苹果GPU、各类边缘设备
- 通用
全球顶会论文80%+用PyTorch实现, 同时支持工业级高并发部署

1. Pytorch总览: How



1. Dataset+DataLoader加载、预处理数据
2. 继承nn.Module定义神经网络模型
3. 定义损失函数、优化器、训练设备
4. 写训练循环, 完成前向传播→反向传播→参数更新
5. 用测试集验证模型效果, 完成推理部署

1. Pytorch总览：适用场景

- 计算机视觉
图像分类、目标检测、分割、生成式 AI
- 自然语言处理
大模型、文本分类、翻译、对话系统
- 音频/语音
语音识别、语音合成、音频分类
- 其它
强化学习、自动驾驶、科学计算等等

1. Pytorch总览：安装

- PyTorch官方个性化安装指令生成页

<https://pytorch.org/get-started/locally/>

PyTorch Build	Stable (2.11.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Pip		LibTorch		Source
Language	Python			C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 7.2	CPU
Run this Command:	<pre>pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu128</pre>				

2. 核心基础：张量定义

张量的定义

PyTorch的运算单元：张量(tensor)，
可以理解为一个多维数组

张量阶数	名称	场景
0	标量	损失值、准确率
1	向量	特征向量、标签
2	矩阵	全连接层权重
3	三阶张量	单张 RGB 图片
n	n阶张量	一批图片、视频帧

Tensor的基本数据类型

32 位浮点型

`torch.FloatTensor`

64 位整型

`torch.LongTensor`

32 位整型

`torch.IntTensor`

16 位整型

`torch.ShortTensor`

64 位浮点型

`torch.DoubleTensor`

在新版Pytorch中，推荐
用dtype参数指定类型

2. 核心基础：张量初始化

- 直接从数据

张量可以直接从数据创建。数据类型会自动推断

```
data = [[1, 2],[3, 4]]  
x_data = torch.tensor(data)
```

- 从NumPy数组

张量可以从NumPy数组创建（反之亦然）

```
np_array = np.array(data)  
x_np = torch.from_numpy(np_array)
```

2. 核心基础：张量初始化

- 从另一个张量

新的张量保留了参数张量的属性（形状、数据类型），除非显式覆盖

```
x_ones = torch.ones_like(x_data) # retains the properties of x_data
print(f"Ones Tensor: \n {x_ones} \n")
```

```
x_rand = torch.rand_like(x_data, dtype=torch.float) # overrides the datatype of x_data
print(f"Random Tensor: \n {x_rand} \n")
```

- 输出

```
Ones Tensor:
tensor([[1, 1],
        [1, 1]])
```

```
Random Tensor:
tensor([[0.5513, 0.5079],
        [0.8693, 0.0571]])
```

2. 核心基础：张量初始化

- 使用随机或常量值

shape是张量维度的元组。在下面的函数中，它决定了输出张量的维度

```
shape = (2,3,)
rand_tensor = torch.rand(shape)
ones_tensor = torch.ones(shape)
zeros_tensor = torch.zeros(shape)

print(f"Random Tensor: \n {rand_tensor} \n")
print(f"Ones Tensor: \n {ones_tensor} \n")
print(f"Zeros Tensor: \n {zeros_tensor}")
```

- 输出

Random Tensor:

```
tensor([[[0.2678, 0.2952, 0.8638],
         [0.5615, 0.7312, 0.6692]]])
```

Ones Tensor:

```
tensor([[[1., 1., 1.],
         [1., 1., 1.]])
```

Zeros Tensor:

```
tensor([[[0., 0., 0.],
         [0., 0., 0.]])
```

2. 核心基础：张量属性

- 张量属性描述了它们的形状、数据类型以及它们存储的设备。

```
tensor = torch.rand(3,4)
```

```
print(f"Shape of tensor: {tensor.shape}")
```

```
print(f"Datatype of tensor: {tensor.dtype}")
```

```
print(f"Device tensor is stored on: {tensor.device}")
```

- 输出

```
Shape of tensor: torch.Size([3, 4])
```

```
Datatype of tensor: torch.float32
```

```
Device tensor is stored on: cpu
```

2. 核心基础：张量操作

- 超过 1200 个张量操作，包括算术、线性代数、矩阵操作（转置、索引、切片）、采样等
- 标准的NumPy风格的索引和切片
- 输出

```
tensor = torch.ones(4, 4)

print(f'First row: {tensor[0]}')
print(f'First column: {tensor[:, 0]}')
print(f'Last column: {tensor[..., -1]}')

tensor[:,1] = 0

print(tensor)
```

```
First row: tensor([1., 1., 1., 1.])
First column: tensor([1., 1., 1., 1.])
Last column: tensor([1., 1., 1., 1.])
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

2. 核心基础：张量操作

- 连接张量

```
t1 = torch.cat([tensor, tensor, tensor], dim=1)
print(t1)
```

- 输出

```
tensor([[1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.]])
```

2. 核心基础：张量操作

- 算数运算

```
y1 = tensor @ tensor.T  
y2 = tensor.matmul(tensor.T)  
  
y3 = torch.rand_like(y1)  
torch.matmul(tensor, tensor.T, out=y3)
```

```
z1 = tensor * tensor  
z2 = tensor.mul(tensor)
```

```
z3 = torch.rand_like(tensor)  
torch.mul(tensor, tensor, out=z3)
```

- 输出

```
tensor([[[1., 0., 1., 1.],  
         [1., 0., 1., 1.],  
         [1., 0., 1., 1.],  
         [1., 0., 1., 1.]])
```

2. 核心基础：张量设备管理

- 张量和模型必须在同一个设备上才能运算

```
device = "cuda" if torch.cuda.is_available() else "cpu"  
x = torch.tensor([1,2,3]).to(device)
```

```
print(x.device)
```

3. 核心模块：构建神经网络

- torch.nn

PyTorch专门为神经网络封装的模块，所有网络层、损失函数、容器都在这里

- 神经网络由对数据执行操作的层/模块组成，其核心组件：

基类：nn.Module

网络层：nn.Linear、nn.Conv2d、nn.Flatten等

激活函数：nn.ReLU、nn.Softmax等

损失函数：nn.CrossEntropyLoss、nn.MSELoss等

容器：nn.Sequential

3.核心模块：构建神经网络

- 获取训练设备

```
device = torch.accelerator.current_accelerator().type \
        if torch.accelerator.is_available() else " cpu "
```

```
print(f"Using {device} device")
```

- 输出

```
Using cuda device
```

```
# 这里假设当前设备CUDA可用
```

3.核心模块：构建神经网络

- 定义类

通过继承nn.Module来定义神经网络，并在__init__中初始化神经网络层。
每个nn.Module子类都在forward方法中实现对输入数据的操作。

```
class NeuralNetwork(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.flatten = nn.Flatten()  
        self.linear_relu_stack = nn.Sequential(  
            nn.Linear(28*28, 512),  
            nn.ReLU(),  
            nn.Linear(512, 512),  
            nn.ReLU(),  
            nn.Linear(512, 10),  
        )  
    def forward(self, x):  
        x = self.flatten(x)  
        logits = self.linear_relu_stack(x)  
        return logits
```

3.核心模块：构建神经网络

- 创建一个NeuralNetwork实例，将其移动到device上，并打印其结构

```
model = NeuralNetwork().to(device)
print(model)
```

- **输出**

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

3.核心模块：构建神经网络

- 模型层

分解 FashionMNIST 模型中的各层，取一个包含 3 张 28×28 图像的示例小批量 (minibatch)，看看它在通过网络时会发生什么

```
input_image = torch.rand(3,28,28)
print(input_image.size())
```

- 输出

```
torch.Size([3, 28, 28])
```

3.核心模块：构建神经网络

- `nn.Flatten`

初始化`nn.Flatten`层，将每个二维的 28×28 图像转换为包含784个像素值的连续数组（保持 `dim=0` 处的小批量维度）

```
flatten = nn.Flatten()
flat_image = flatten(input_image)
print(flat_image.size())
```

- 输出

```
torch.Size([3, 784])
```

3.核心模块：构建神经网络

- **nn.Linear**

线性层，使用其存储的权重和偏置对输入进行线性变换

```
layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())
```

- **输出**

```
torch.Size([3, 20])
```

3.核心模块：构建神经网络

- nn.ReLU

在各线性层之间使用ReLU激活函数

```
print(f'Before ReLU: {hidden1}\n\n")
hidden1 = nn.ReLU()(hidden1)
print(f'After ReLU: {hidden1}')
```

- 输出

```
Before ReLU: tensor([[[ 0.4812, -0.1855, -0.5604, -0.5288,  0.1046, -0.6144, -0.0194,  0.5290,
  0.0763,  0.5170, -0.3571,  0.2859,  0.3366,  0.5851,  0.0283,  0.1084,
  0.2790, -0.1351,  0.4640, -0.4960],
 [ 0.1443, -0.3172, -0.0686, -0.2285, -0.1671, -0.0913,  0.0768,  1.0552,
  0.0984,  0.5287, -0.7454, -0.0322,  0.0910,  0.2811,  0.3954,  0.3928,
  0.3221, -0.1196,  0.1975, -0.5049],
 [ 0.3634, -0.2807, -0.3782, -0.1640,  0.1847, -0.3414,  0.1158,  1.2596,
 -0.2215,  0.6581, -0.6830,  0.0055,  0.1086,  0.6827, -0.1997,  0.1416,
  0.2429, -0.4890,  0.2843, -0.4609]], grad_fn=<AddmmBackward0>)
```

```
After ReLU: tensor([[[0.4812, 0.0000, 0.0000, 0.0000, 0.1046, 0.0000, 0.0000, 0.5290, 0.0763,
  0.5170, 0.0000, 0.2859, 0.3366, 0.5851, 0.0283, 0.1084, 0.2790, 0.0000,
  0.4640, 0.0000],
 [0.1443, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0768, 1.0552, 0.0984,
  0.5287, 0.0000, 0.0000, 0.0910, 0.2811, 0.3954, 0.3928, 0.3221, 0.0000,
  0.1975, 0.0000],
 [0.3634, 0.0000, 0.0000, 0.0000, 0.1847, 0.0000, 0.1158, 1.2596, 0.0000,
  0.6581, 0.0000, 0.0055, 0.1086, 0.6827, 0.0000, 0.1416, 0.2429, 0.0000,
  0.2843, 0.0000]], grad_fn=<ReluBackward0>)
```

3.核心模块：构建神经网络

- **nn.Sequential**

nn.Sequential是一个有序模块容器。数据按照定义的不同顺序通过所有模块

```
seq_modules = nn.Sequential(  
    flatten,  
    layer1,  
    nn.ReLU(),  
    nn.Linear(20, 10)  
)
```

```
input_image = torch.rand(3,28,28)  
logits = seq_modules(input_image)
```

3.核心模块：构建神经网络

- `nn.Softmax`

模型对每个类别的预测概率

```
softmax = nn.Softmax(dim=1)  
pred_prob = softmax(logits)
```

3.核心模块：构建神经网络

- 模型参数

神经网络内部的许多层都是参数化的，即具有在训练期间优化的相关权重和偏置继承nn.Module会自动跟踪模型对象内定义的所有字段，并使所有参数都可以通过模型的parameters()或named_parameters()方法访问

```
print(f"Model structure: {model}\n\n")
```

```
for name, param in model.named_parameters():  
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

3.核心模块：构建神经网络

- 输出

```
Model structure: NeuralNetwork(  
  (flatten): Flatten(start_dim=1, end_dim=-1)  
  (linear_relu_stack): Sequential(  
    (0): Linear(in_features=784, out_features=512, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=512, out_features=512, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=512, out_features=10, bias=True)  
  )  
)
```

```
Layer: linear_relu_stack.0.weight | Size: torch.Size([512, 784]) | Values :  
tensor([[[-0.0158, 0.0088, 0.0175, ..., 0.0318, 0.0024, 0.0175],  
        [-0.0138, -0.0299, -0.0257, ..., -0.0250, 0.0036, 0.0299]],  
        device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.0.bias | Size: torch.Size([512]) | Values : tensor([-  
0.0110, -0.0255], device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.2.weight | Size: torch.Size([512, 512]) | Values :  
tensor([[ 0.0076, -0.0259, 0.0349, ..., -0.0224, -0.0156, -0.0226],  
        [-0.0389, -0.0201, 0.0370, ..., -0.0055, -0.0267, -0.0304]],  
        device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.2.bias | Size: torch.Size([512]) | Values :  
tensor([0.0254, 0.0122], device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.4.weight | Size: torch.Size([10, 512]) | Values :  
tensor([[ 0.0088, -0.0188, -0.0149, ..., -0.0250, 0.0217, 0.0397],  
        [ 0.0042, 0.0350, -0.0065, ..., -0.0095, -0.0337, -0.0142]],  
        device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.4.bias | Size: torch.Size([10]) | Values : tensor([-  
0.0266, -0.0178], device='cuda:0', grad_fn=<SliceBackward0>)
```

4. 核心机制：自动求导

- 定义

Autograd是PyTorch的自动微分引擎，自动完成反向传播的梯度计算，无需手动写求导公式，是深度学习训练的核心

- 考虑最简单的单层神经网络，输入 x ，参数 w 和 b ，以及一些损失函数。

```
import torch
```

```
x = torch.ones(5) # input tensor
```

```
y = torch.zeros(3) # expected output
```

```
w = torch.randn(5, 3, requires_grad=True)
```

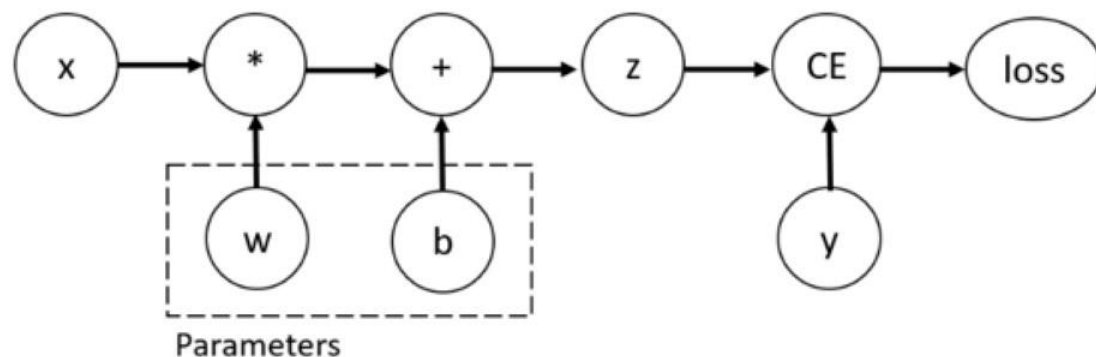
```
b = torch.randn(3, requires_grad=True)
```

```
z = torch.matmul(x, w)+b
```

```
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

4. 核心机制：自动求导

- 这段代码定义了以下计算图



- 在这个网络中， w 和 b 是参数，需要进行优化。
- 因此，需要能够计算损失函数相对于这些变量的梯度。
- 为此，将这些张量的`requires_grad`属性设置为 `True`。

4. 核心机制：自动求导

- 计算梯度

为了优化神经网络中参数的权重，需要计算损失函数相对于参数的导数

```
loss.backward()  
print(w.grad)  
print(b.grad)
```

- 输出

```
tensor([[[0.0651, 0.0246, 0.3265],  
         [0.0651, 0.0246, 0.3265],  
         [0.0651, 0.0246, 0.3265],  
         [0.0651, 0.0246, 0.3265],  
         [0.0651, 0.0246, 0.3265]])  
tensor([0.0651, 0.0246, 0.3265])
```

4. 核心机制：自动求导

- 禁用梯度跟踪

当只想通过网络进行前向计算时，可以通过使用 `torch.no_grad()` 块来停止跟踪计算

```
z = torch.matmul(x, w)+b  
print(z.requires_grad)
```

```
with torch.no_grad():  
    z = torch.matmul(x, w)+b  
print(z.requires_grad)
```

- 输出

True

False

4. 核心机制：自动求导

- 优化模型参数

超参数

```
learning_rate = 1e-3
```

```
batch_size = 64
```

```
epochs = 5
```

损失函数

```
# Initialize the loss function
```

```
loss_fn = nn.CrossEntropyLoss()
```

4. 核心机制：自动求导

- 优化模型参数

- 优化器

通过注册需要训练的模型参数并传递学习率超参数来初始化优化器

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

- Train_loop

```
def train_loop(dataloader, model, loss_fn, optimizer):  
    size = len(dataloader.dataset)  
    # Set the model to training mode - important for batch  
    normalization and dropout layers  
    # Unnecessary in this situation but added for best practices  
    model.train()  
    for batch, (X, y) in enumerate(dataloader):  
        # Compute prediction and loss  
        pred = model(X)  
        loss = loss_fn(pred, y)  
  
        # Backpropagation  
        loss.backward()  
        optimizer.step()  
        optimizer.zero_grad()  
  
        if batch % 100 == 0:  
            loss, current = loss.item(), batch *  
            batch_size + len(X)
```

5. 总结: PyTorch常用功能

1. 基础张量 (Tensor)

导入

```
import torch
```

创建

```
torch.tensor([1, 2, 3])
torch.zeros(2, 3)
torch.ones(2, 3)
torch.randn(2, 3) # 服从正态分布
torch.arange(0, 10, 2)
torch.linspace(0, 1, steps=5)
torch.eye(3) # 单位矩阵
```

属性

```
x.shape # 形状
x.dtype # 数据类型
x.device # 设备
x.dim() # 维度
```

操作

```
x = x * y - x * y, x * y, x / y
x @ y # 矩阵乘法
x.t() / x.T # 转置
x.reshape(2, -1) # reshape
x.view(2, -1) # 需连续内存
x.sum(dim=0) # 按维度求和
x.mean(dim=1) # 按维度求平均
x.max(dim=0) # 返回 (values, indices)
x[1] , x[:, 1] # 索引与切片
```

4. 数据处理 (Data)

Dataset

```
from torch.utils.data import Dataset
class MyDataset(Dataset):
    def __init__(self): ...
    def __len__(self):
        return ...
    def __getitem__(self, idx):
        return data, label
```

DataLoader

```
from torch.utils.data import DataLoader
loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4,
    drop_last=False,
)
```

常用采样器

```
torch.utils.data.RandomSampler
torch.utils.data.SequentialSampler
torch.utils.data.DistributedSampler
```

2. 数据类型 & 设备

数据类型

```
torch.float32 (默认)
torch.float64
torch.int64
torch.bool
```

类型转换

```
x.float() x.long() x.double()
x.to(dtype=torch.float32)
```

设备 (Device)

```
torch.device('cpu')
torch.device('cuda') # 需安装 CUDA
```

移动张量

```
x.to(device)
x.cuda() # 等价于 x.to('cuda')
x.cpu()
```

⚠ 小贴士

尽量在创建张量时指定 device, 避免频繁的数据拷贝。

5. 神经网络 (nn)

定义模型

```
import torch.nn as nn
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 1)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = self.fc(x)
        return self.relu(x)
```

常用层

```
nn.Linear(in, out)
nn.Conv2d(in, out, kernel_size, stride=1, padding=0)
nn.ReLU(), nn.Sigmoid(), nn.Tanh()
nn.BatchNorm2d(num_features)
nn.Dropout(p=0.5)
nn.MaxPool2d(kernel_size)
nn.Flatten(), nn.AdaptiveAvgPool2d(1)
```

模型工具

```
model = Net()
model.train() # 训练模式
model.eval() # 评估模式
for p in model.parameters(): ...
model.to(device)
```



3. 自动求导 (Autograd)

张量

```
x = torch.tensor(2.0, requires_grad=True)
y = x * x * 3 * x
y.backward() # 反向传播
x.grad # dy/dx
```

梯度控制

```
for param in model.parameters():
    param.requires_grad = False
```

上下文管理

```
with torch.no_grad():
    # 不记录梯度

with torch.enable_grad():
    # 恢复记录
```

detach

```
y = x.detach() # 从计算图中分离
```

⚠ 小贴士

只有标量 (0-dim) 或经 .sum() 等操作后的结果才能直接 .backward()。

7. 训练流程 (典型)

```
model = Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, 10, 0.1)

for epoch in range(epochs):
    model.train()
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        scheduler.step()
    model.eval()
    with torch.no_grad():
        for inputs, targets in val_loader:
            outputs = model(inputs.to(device))
            # 计算指标 ...
```



8. 常用工具

保存与加载

```
torch.save(model, 'file.pth')
obj = torch.load('file.pth', map_location='cpu')
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))
```

可视化

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("./logs")
writer.add_scalar("Loss/train", loss, step)
writer.add_graph(model, example_input)
writer.close()
```

其他

```
torch.nn.functional.softmax # 返回概率分布
torch.backends.cuda.is_available() # 是否支持 CUDA
torch.cuda.is_available() # 是否支持 GPU
```

9. 常用函数速查

张量操作

```
torch.cat(tensors, dim)
torch.stack(tensors, dim=0)
torch.chunk(tensor, chunks, dim)
torch.split(tensor, split_size, dim)
torch.split_with_sizes(tensors, sizes, dim)
torch.reshape(tensor, shape)
torch.view_as_flatten(tensors)
```

数学运算

```
torch.exp() torch.log()
torch.exp2() torch.log2()
torch.clamp(x, min, max)
torch.clamp_min(x, min)
torch.clamp_max(x, max)
torch.addmm(a, b, c)
torch.addmv(a, b, c)
torch.addvmm(a, b, c)
```

张量与张量

```
torch.add(a, b)
torch.add_!(a, b)
torch.addmm(a, b, c)
torch.addmv(a, b, c)
torch.addvmm(a, b, c)
torch.addbmm(a, b, c)
torch.addcmul(a, b, c)
torch.addcdiv(a, b, c)
torch.addcdiv_!(a, b, c)
torch.addcmul_!(a, b, c)
torch.addcdiv_!(a, b, c)
```

集合

```
torch.setdiff1d(a, b)
torch.setdiff2d(a, b)
torch.setdiff3d(a, b)
torch.setdiff4d(a, b)
torch.setdiff5d(a, b)
torch.setdiff6d(a, b)
torch.setdiff7d(a, b)
torch.setdiff8d(a, b)
torch.setdiff9d(a, b)
torch.setdiff10d(a, b)
torch.setdiff11d(a, b)
torch.setdiff12d(a, b)
torch.setdiff13d(a, b)
torch.setdiff14d(a, b)
torch.setdiff15d(a, b)
torch.setdiff16d(a, b)
torch.setdiff17d(a, b)
torch.setdiff18d(a, b)
torch.setdiff19d(a, b)
torch.setdiff20d(a, b)
torch.setdiff21d(a, b)
torch.setdiff22d(a, b)
torch.setdiff23d(a, b)
torch.setdiff24d(a, b)
torch.setdiff25d(a, b)
torch.setdiff26d(a, b)
torch.setdiff27d(a, b)
torch.setdiff28d(a, b)
torch.setdiff29d(a, b)
torch.setdiff30d(a, b)
torch.setdiff31d(a, b)
torch.setdiff32d(a, b)
torch.setdiff33d(a, b)
torch.setdiff34d(a, b)
torch.setdiff35d(a, b)
torch.setdiff36d(a, b)
torch.setdiff37d(a, b)
torch.setdiff38d(a, b)
torch.setdiff39d(a, b)
torch.setdiff40d(a, b)
torch.setdiff41d(a, b)
torch.setdiff42d(a, b)
torch.setdiff43d(a, b)
torch.setdiff44d(a, b)
torch.setdiff45d(a, b)
torch.setdiff46d(a, b)
torch.setdiff47d(a, b)
torch.setdiff48d(a, b)
torch.setdiff49d(a, b)
torch.setdiff50d(a, b)
torch.setdiff51d(a, b)
torch.setdiff52d(a, b)
torch.setdiff53d(a, b)
torch.setdiff54d(a, b)
torch.setdiff55d(a, b)
torch.setdiff56d(a, b)
torch.setdiff57d(a, b)
torch.setdiff58d(a, b)
torch.setdiff59d(a, b)
torch.setdiff60d(a, b)
torch.setdiff61d(a, b)
torch.setdiff62d(a, b)
torch.setdiff63d(a, b)
torch.setdiff64d(a, b)
torch.setdiff65d(a, b)
torch.setdiff66d(a, b)
torch.setdiff67d(a, b)
torch.setdiff68d(a, b)
torch.setdiff69d(a, b)
torch.setdiff70d(a, b)
torch.setdiff71d(a, b)
torch.setdiff72d(a, b)
torch.setdiff73d(a, b)
torch.setdiff74d(a, b)
torch.setdiff75d(a, b)
torch.setdiff76d(a, b)
torch.setdiff77d(a, b)
torch.setdiff78d(a, b)
torch.setdiff79d(a, b)
torch.setdiff80d(a, b)
torch.setdiff81d(a, b)
torch.setdiff82d(a, b)
torch.setdiff83d(a, b)
torch.setdiff84d(a, b)
torch.setdiff85d(a, b)
torch.setdiff86d(a, b)
torch.setdiff87d(a, b)
torch.setdiff88d(a, b)
torch.setdiff89d(a, b)
torch.setdiff90d(a, b)
torch.setdiff91d(a, b)
torch.setdiff92d(a, b)
torch.setdiff93d(a, b)
torch.setdiff94d(a, b)
torch.setdiff95d(a, b)
torch.setdiff96d(a, b)
torch.setdiff97d(a, b)
torch.setdiff98d(a, b)
torch.setdiff99d(a, b)
```

随机数

```
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.rand(n)
torch.randn(n)
torch.randn_like(x)
torch.randperm(n)
torch.randperm(n, dtype=torch.long)
```

6. 损失函数 & 优化器

损失函数

```
nn.MSELoss()
nn.CrossEntropyLoss() # 分类 (含 Softmax)
nn.BCEWithLogitsLoss() # 二分类 (含 Sigmoid)
nn.L1Loss()
```

优化器

```
torch.optim.SGD(params, lr, momentum=0.9)
torch.optim.Adam(params, lr, weight_decay=0)
torch.optim.RMSprop(params, lr)
```

学习率调度器

```
torch.optim.lr_scheduler.StepLR(optimizer, step_size, gamma)
torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max)
```

⚠ 小贴士

查看所有可用的损失函数与优化器：
<https://pytorch.org/docs/stable/>

5. 拓展：案例和资料

- 拓展学习案例

官方实例教程：https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html

中文入门教程：<https://mofanpy.com/tutorials/machine-learning/torch/>

开源实战项目：<https://github.com/xiaotudui/pytorch-tutorial>

- Pytorch常用速查工具

官方速查表：<https://pytorch-cn.com/tutorials/beginner/ptcheat.html>

完整 API 速查表：https://www.learnpytorch.io/pytorch_cheatsheet/

PDF 版速查表：<https://www.mad.tf.fau.de/files/2019/07/pytorch-cheatsheet-en.pdf>